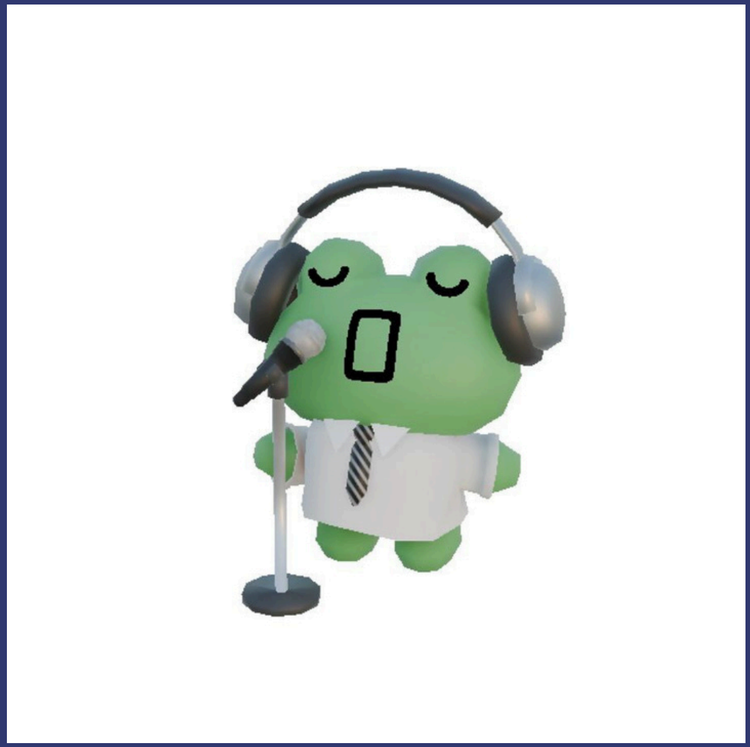
26T1 Week02

Object-Oriented Design & Abstraction

Software Design & Architecture



Annelise

agenda

- super vs this
- Abstract Classes vs Interfaces
- static
- JavaDoc
- Inheritance & Polymorphism
- .toString() and .equals()
- Access Modifiers
- + Code Demos!

super vs. this

- The difference between these two is what classes they refer to.
- *super* refers to the parent class (if any), and *this* refers to the class itself.
- When we call super(...), we actually call the appropriate constructor of the parent class.
- Likewise, if we call this(...), we call the appropriate constructor of the current class.

```java
public class Parent {
  private String a;
  private String b;

  public Parent(String a, String b) {
    this.a = a;
    this.b = b;
  }
}

public class Child extends Parent {
  private String c;
  public Child(String a, String b, String c) {
    // calls the parent class!
    super(a, b);
    // this refers to itself, so it's setting the
    // private String c variable!
    this.c = c;
  }
}
```

abstract classes vs. interfaces

Abstract classes can’t be instantiated along with Interfaces. Both can be used to model classes, so what’s the difference?

abstract class ✸

- **Can’t be instantiated**
- Allow **common properties** and **abstract methods**.
- Classes **extend one abstract class**.

---

interface ✦

- Define **method signatures only**.
- **No shared** state/properties (only constant static variables).
- Classes can **implement many interfaces**.

```java
public abstract class Plane {
  private String name;
  private String engine;

  public Plane(String name, String engine) {
    this.name = name;
    this.engine = engine;
  }

  public abstract void fly();
}


public class Jet extends Plane {
  public Jet(String name, String engine) {
    super(name, engine);
  }
  // fly() is overrode in the original parent abstract
class
  @Override
  public void fly() {
    System.out.println("Plane flying!");
  }
}
```

```java
public interface IFlyable {
  void fly();
}

public class Helicopter implements IFlyable {
  // we can also call fly() on Helicopter, but it won't
have a name or engine.
  @Override
  public void fly() {
    System.out.println("Helicopter flying!");
  }
}
```

what it looks like

Interfaces define **capabilities**.
(“Can this class do X?”)

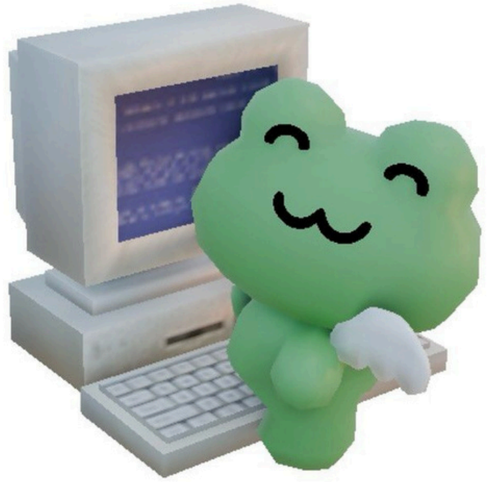
Abstract classes define **families**.
(“Is this class a kind of Y?”)

rule of thumb

static keyword

- Static variables are **attached to the class declaration** itself, **not the instance**.
  - Kinda like **global variables** for a class.
- Instances of classes can interact with static variables and modify them.

```java
public abstract class Animal {

  public static int COUNTER = 0;

  public Animal() {
    COUNTER++;
    ...
  }
}

public class Cat extends Animal {
  public Cat() {
    ...
  }
}

public class Dog extends Animal {
  public Dog() {
    ...
  }
}
```



code review

Shapes

inheritance & polymorphism

```java
class Animal {
    void speak() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    @Override
    void speak() {
        System.out.println("Woof!");
    }
}

class Cat extends Animal {
    @Override
    void speak() {
        System.out.println("Meow!");
    }
}
```

```java
class Calculator {
    // Method overloading (compile-time polymorphism)
    int add(int a, int b) {
        return a + b;
    }
    double add(double a, double b) {
        return a + b;
    }
}
```

- In inheritance, child subclasses will inherit the properties and methods of parent classes (unless specified).
- Polymorphism is the ability for a single method/interface to take on many forms. This means the same code works differently on different objects.
    - e.g. we can override or overload methods and constructors to provide differing behaviour.

method overloading

- Same method name, **different parameter lists**
- Decided at **compile time**
- Parameters must differ by number, types, or order
- Return type alone cannot distinguish overloads
- Used to provide convenient variations of the same operation

```java
public abstract class Animal {

  public static int COUNTER = 0;

  public Animal() {
    COUNTER++;
    ...
  }
}

public class Cat extends Animal {
  public Cat() {
    ...
  }
}

public class Dog extends Animal {
  public Dog() {
    ...
  }
}
```

method overriding

- Same method signature **(must be exactly the same).** Subclass provides a new implementation of a superclass method;
- Decided at **runtime** (‘dynamic polymorphism’)
- Enables dynamic behaviour based on object type
- **Access level cannot be more restrictive**

```java
public abstract class Animal {

  public static int COUNTER = 0;

  public Animal() {
    COUNTER++;
    ...
  }
}

public class Cat extends Animal {
  public Cat() {
    ...
  }
}

public class Dog extends Animal {
  public Dog() {
    ...
  }
}
```

R is for (over)Riding and Runtime

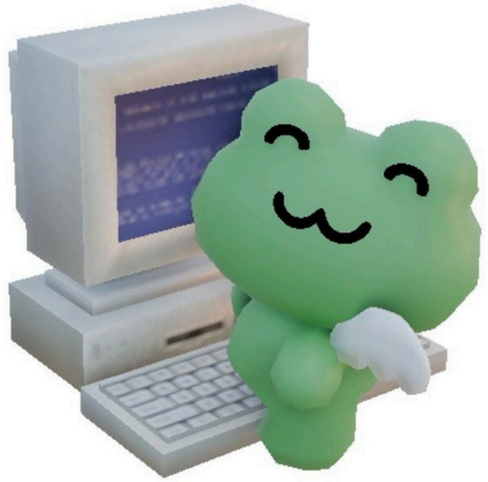
L before R;
overLoading before overRiding;
i.e. overLoading at compile time,
overRiding at run time.

which comes first?

- Generally follows this format
  - A short description, followed by parameter name and return.
- In this course we are generally not going to require that you write JavaDoc, **except** when specified.

JavaDoc

```java
package employee;

public class Employee {
    private String name;
    private int salary;

    /**
     * Creates an Employee with the given name and salary.
     *
     * @param name   The full name of the employee.
     * @param salary The employee's yearly salary in dollars.
     */
    public Employee(String name, int salary) {
        this.name = name;
        this.salary = salary;
    }

    /**
     * Returns the employee's name
     *
     * @return The full name of the employee.
     */
    public String getName() {
        return name;
    }
}
```



code review

Employee

- toString() should return a human-readable description of the object
  - Used automatically when you print an object or concatenate it with a string
  - Should include the key fields and be clear, concise + useful for debugging
- Default version is usually unhelpful, hence the need to override!

```java
public class Employee {
  private String name;
  private int salary;

  public Employee(String name, int salary) {
    this.name = name;
    this.salary = salary;
  }
  ...
  @Override
  public String toString() {
    return getClass().getName() + "[name=" + name + ", salary=" + salary + "]";
  }

  @Override
  public boolean equals(Object obj) {
    if (this == obj)
      return true;
    if (obj == null)
      return false;
    if (getClass() != obj.getClass())
      return false;
    Employee other = (Employee) obj;
    if (name.equals(other.name) && salary == other.salary)
      return true;
    return false;
  }
}
```

.toString()

- Used to test **logical equality**, not memory identity
  - == compares by pointer reference.
- Override it when **two different objects can represent the same value**
- Must be reflexive, symmetric, transitive, consistent, **null-safe**: (i.e. x.equals(null) must return false)
- Usually compares the same fields shown in toString()

```java
public class Employee {
  private String name;
  private int salary;

  public Employee(String name, int salary) {
    this.name = name;
    this.salary = salary;
  }
  ...
  @Override
  public String toString() {
    return getClass().getName() + "[name=" + name + ", salary=" + salary + "]";
  }

  @Override
  public boolean equals(Object obj) {
    if (this == obj)
      return true;
    if (obj == null)
      return false;
    if (getClass() != obj.getClass())
      return false;
    Employee other = (Employee) obj;
    if (name.equals(other.name) && salary == other.salary)
      return true;
    return false;
  }
}
```

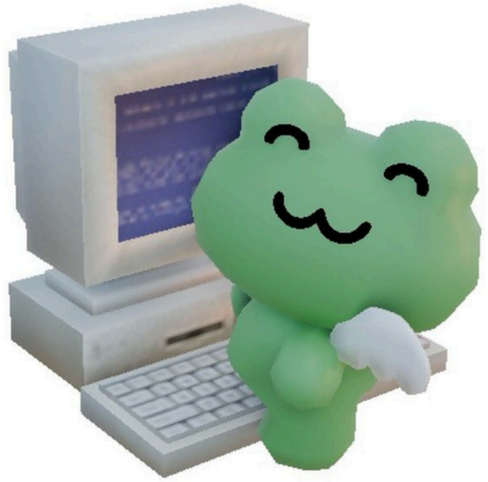
.equals( )

- Avoid instanceof
  - It allows for subtype comparisons, i.e. different subclasses could be considered equal
- Prefer getClass()
  - Checks the exact runtime type, ensuring both objects are the same class
  - Keeps equals() predictable and consistent + preserves the equals() contract

```java
public boolean equals(Object obj) {
  // Check that the passed in object is not null.
  if (object == null) return false;

  // Check if the passed in object is the same instance as the calling object.
  if (this == object) return true;

  // Check the concrete type of the calling object matches the concrete type of the passed in object.

  if (!this.getClass().equals(object.getClass())) return false;

  // Note this is the robust way to allow subclasses to invoke super.equals().
}
```

.equals()

(some guidelines)



code review

Employee (pt. 2), Access

Modifier	Class	Package	Subclass	Global
Public	✓	✓	✓	✓
Protected	✓	✓	✓	✗
Default	✓	✓	✗	✗
Private	✓	✗	✗	✗

access modifiers

lab time!

SitarME306